

Below is a complete **Part 5 Report** in the same style and structure as your previous reports. You can copy it directly into Microsoft Word, add your screenshots in the respective sections, and replace the GitHub repository links with your actual links.

Google Cloud Security Engineering Lab

Part 5 – GitOps Continuous Delivery Using Cloud Build & Google Kubernetes Engine

Prepared By

Senthamizhan Mathew J

Google Cloud Security Engineering Portfolio

Technologies Used

- Google Cloud Platform (GCP)
- Google Kubernetes Engine (GKE)
- Cloud Build
- GitHub
- GitHub App
- Docker
- Artifact Registry
- Kubernetes
- YAML
- Cloud Shell
- Cloud Build Triggers

Table of Contents

1. Introduction
2. Objectives
3. Architecture Diagram
4. UML Architecture Diagram
5. Project Workflow
6. Environment Setup

7. Implementation Steps
8. Results
9. Challenges Faced
10. Learning Outcomes
11. Conclusion
12. Source Code
13. README.md
14. Lessons Learned

1. Introduction

GitOps is a modern Continuous Delivery methodology where Git repositories serve as the single source of truth for application deployment. Any change committed to the repository automatically triggers an automated deployment pipeline.

This project demonstrates the implementation of a GitOps Continuous Delivery pipeline using Google Cloud Build, GitHub repositories, Google Kubernetes Engine, Docker containers, Artifact Registry, and Kubernetes deployment manifests. Cloud Build monitors GitHub repositories for changes and automatically builds and deploys applications to Kubernetes.

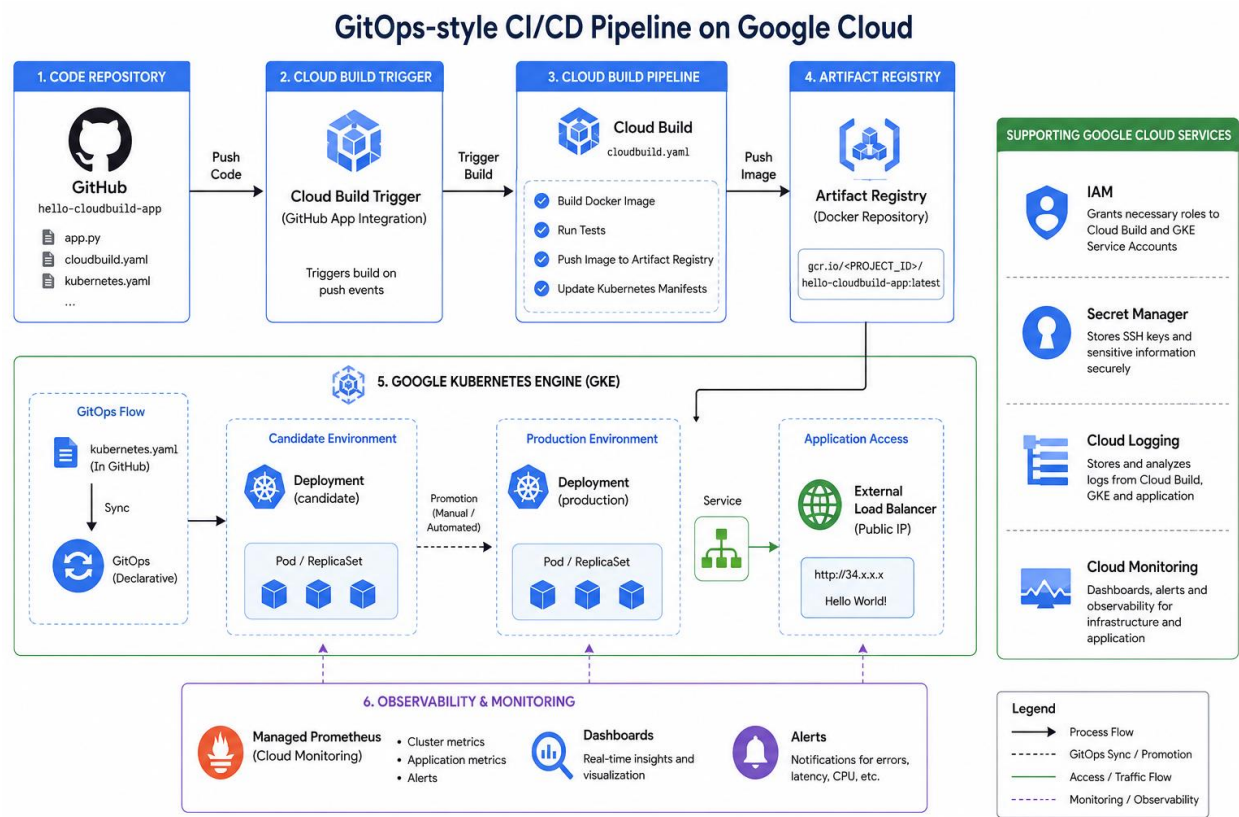
2. Objectives

The objectives of this project are:

- Create GitHub repositories for application and deployment.
- Configure Cloud Build.
- Configure Cloud Build GitHub App.
- Create Cloud Build Trigger.
- Configure cloudbuild.yaml.
- Build Docker images automatically.
- Push Docker images into Artifact Registry.
- Deploy application to Google Kubernetes Engine.
- Automate Continuous Integration.
- Automate Continuous Delivery.
- Validate successful deployment.

3. Architecture Diagram

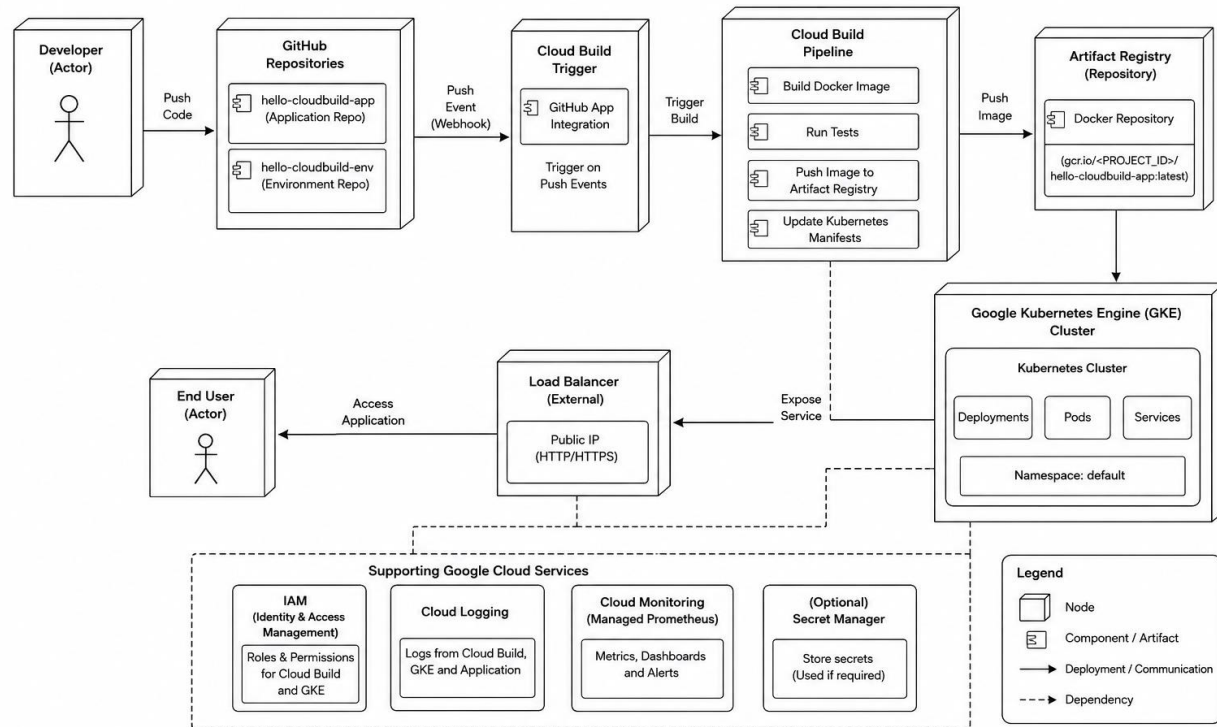
Figure 3.1 – GitOps Continuous Delivery Architecture



4. UML Architecture Diagram

Figure 4.1 – UML Deployment Diagram

GitOps-style CI/CD Pipeline on Google Cloud – UML Deployment Diagram



5. Project Workflow

The implementation followed the workflow below.

1. Configure Google Cloud Project.
2. Create Artifact Registry Repository.
3. Create Google Kubernetes Engine Cluster.
4. Configure GitHub Authentication.
5. Create GitHub Application Repository.
6. Create GitHub Environment Repository.
7. Upload Application Source Code.
8. Upload Kubernetes Deployment Files.
9. Configure Cloud Build Trigger.
10. Connect GitHub Repository.
11. Push Source Code.
12. Trigger Cloud Build Pipeline.
13. Build Docker Image.
14. Push Docker Image to Artifact Registry.
15. Deploy Application to GKE.

16. Verify Successful Deployment.

6. Environment Setup

Component	Description
Cloud Platform	Google Cloud Platform
Source Control	GitHub
CI/CD Tool	Cloud Build
Container Registry	Artifact Registry
Compute Platform	Google Kubernetes Engine
Container Runtime	Docker
Deployment	Kubernetes
Configuration	YAML

7. Implementation

Step 1 – Configure Google Cloud Environment

Description

Configured the Google Cloud project and enabled all required APIs.

```
CLOUD SHELL
Terminal (qwiklabs-gcp-04-20bd7dffd629) x + v

Downloading gh from
https://github.com/cli/cli/releases/download/v2.95.0/gh_2.95.0_linux_amd64.tar.gz
Saved as ~/Downloads/web1/gh/2.95.0/gh_2.95.0_linux_amd64.tar.gz
Extracting ~/Downloads/web1/gh/2.95.0/gh_2.95.0_linux_amd64.tar.gz
Installing to ~/.local/opt/gh-v2.95.0/bin/gh

Installed 'gh v2.95.0' as ~/.local/bin/gh

Edit ~/.config/envman/PATH.env to add:
~/.local/bin

>>> ACTION REQUIRED <<<
Copy, paste & run the following command:
source ~/.config/envman/PATH.env
(newly opened terminal windows will update automatically)

? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 56F3-8296
Press Enter to open https://github.com/login/device in your browser...
https://github.com/login/device

✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
! Authentication credentials saved in plain text
✓ Logged in as senthamizhan-matthew
senthamizhan-matthew
student-00-ebd648e9e063@qwiklabs.net
student_00_ebd648e9e063@cloudshell:~ (qwiklabs-gcp-04-20bd7dffd629)$
student_00_ebd648e9e063@cloudshell:~ (qwiklabs-gcp-04-20bd7dffd629)$
```



```
CLOUD SHELL
Terminal (qwiklabs-gcp-04-20bd7dffd629) x + v

Downloading gh from
https://github.com/cli/cli/releases/download/v2.95.0/gh_2.95.0_linux_amd64.tar.gz
Saved as ~/Downloads/web1/gh/2.95.0/gh_2.95.0_linux_amd64.tar.gz
Extracting ~/Downloads/web1/gh/2.95.0/gh_2.95.0_linux_amd64.tar.gz
Installing to ~/.local/opt/gh-v2.95.0/bin/gh

Installed 'gh v2.95.0' as ~/.local/bin/gh

Edit ~/.config/envman/PATH.env to add:
~/.local/bin

>>> ACTION REQUIRED <<<
Copy, paste & run the following command:
source ~/.config/envman/PATH.env
(newly opened terminal windows will update automatically)

? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 56F3-8296
Press Enter to open https://github.com/login/device in your browser...
https://github.com/login/device

✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
! Authentication credentials saved in plain text
✓ Logged in as senthamizhan-matthew
senthamizhan-matthew
senthamizhan-matthew
student-00-ebd648e9e063@qwiklabs.net
student_00-ebd648e9e063@cloudshell:~ (qwiklabs-gcp-04-20bd7dffd629)$
student_00-ebd648e9e063@cloudshell:~ (qwiklabs-gcp-04-20bd7dffd629)$
```

Step 5 – Create Application Repository

Description

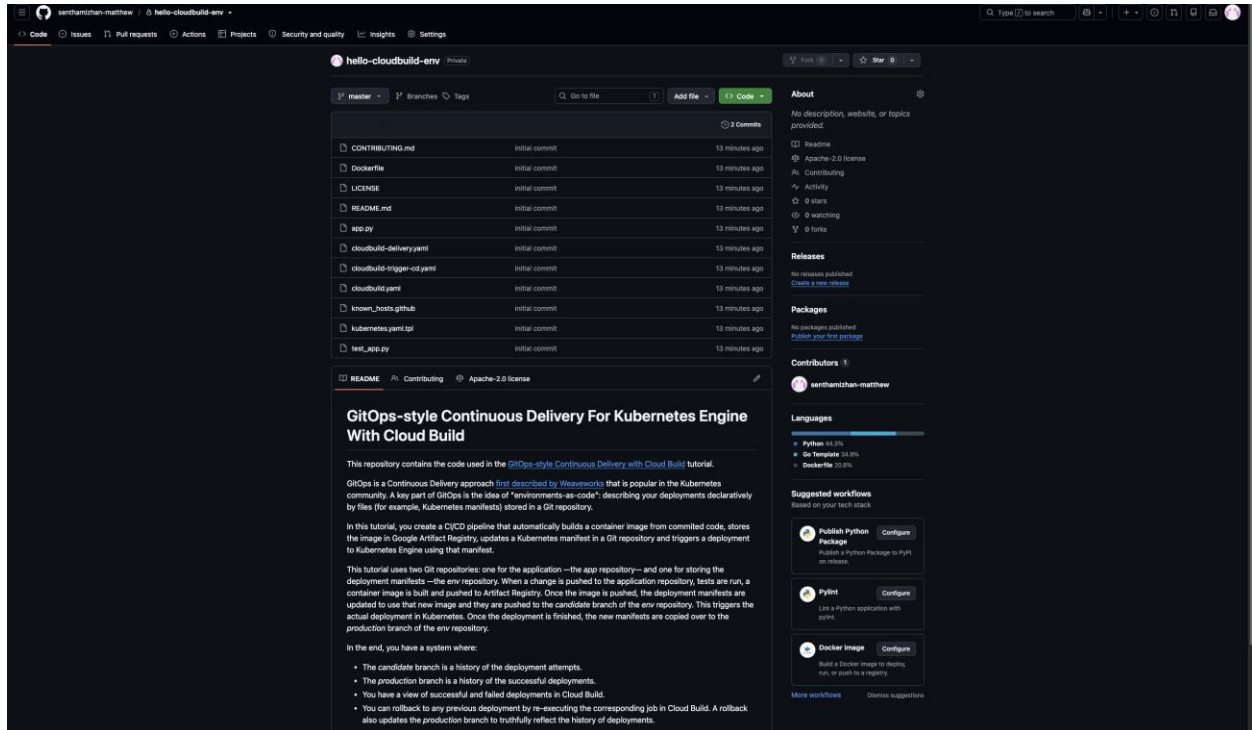
Created the **hello-cloudbuild-app** repository to store the Flask application source code.

The screenshot shows the GitHub repository page for 'hello-cloudbuild-app'. The repository is owned by 'senthamizhan-matthew' and is currently on the 'master' branch. The file list includes 'CONTRIBUTING.md', 'Dockerfile', 'LICENSE', 'README.md', 'app.py', 'cloudBuild-delivery.yaml', 'cloudBuild-trigger-cd.yaml', 'cloudBuild.yaml', 'known_hosts.github', 'kubernetes.yaml.tpl', and 'test_app.py'. The repository description is 'GitOps-style Continuous Delivery For Kubernetes Engine With Cloud Build'. The repository is licensed under the Apache-2.0 license. The repository is currently empty, with no releases or packages published. The repository is currently on the 'master' branch. The repository is currently on the 'master' branch.

Step 6 – Create Environment Repository

Description

Created the **hello-cloudbuild-env** repository to store Kubernetes deployment manifests.



Step 7 – Upload Application Files

Description

Uploaded the Flask application, Dockerfile, Cloud Build configuration, and deployment templates into the GitHub repository.

Step 8 – Upload Deployment Files

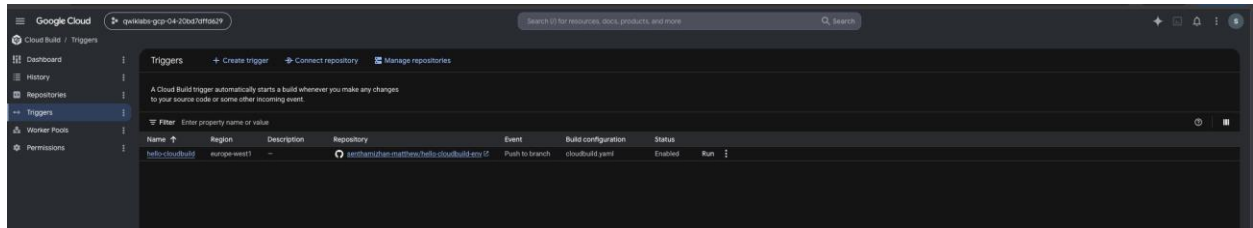
Description

Uploaded Kubernetes deployment manifests into the environment repository.

Step 9 – Configure Cloud Build Trigger

Description

Configured a Cloud Build trigger connected to the GitHub repository using **cloudbuild.yaml**.



Step 10 – Push Source Code

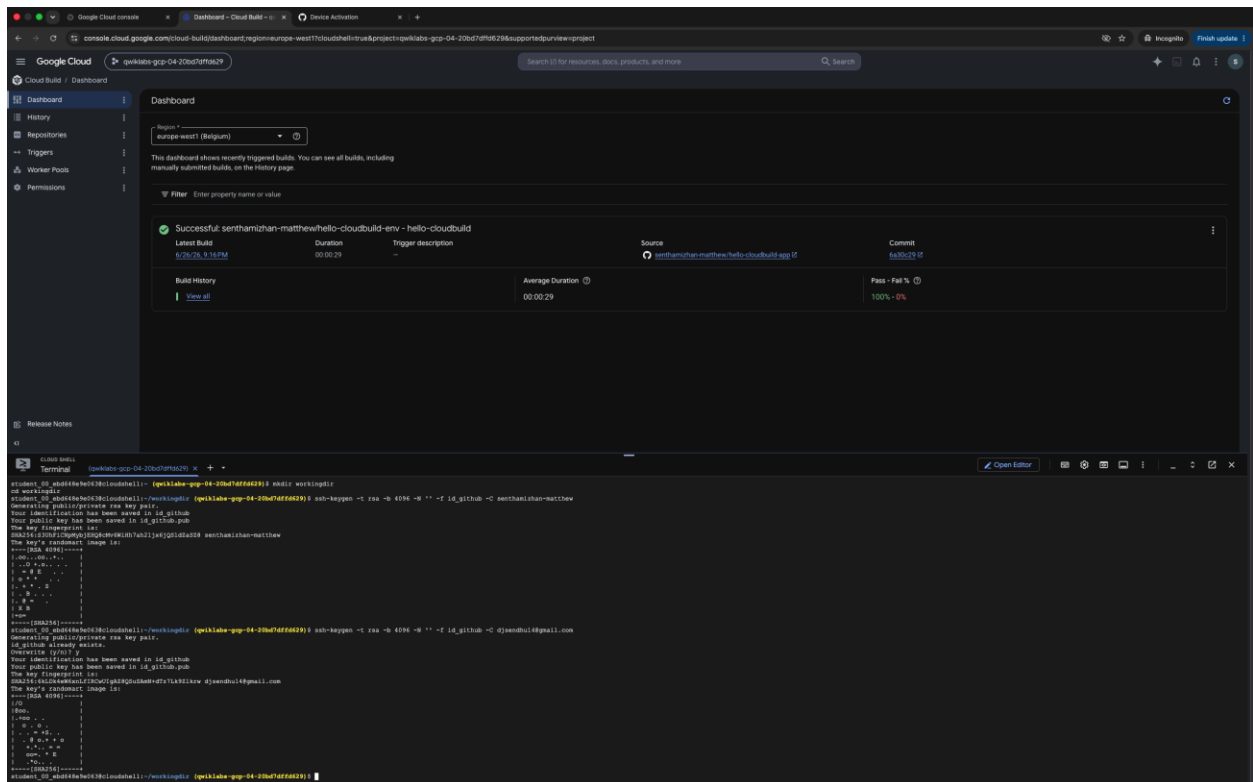
Description

Committed and pushed the latest application source code to GitHub, automatically triggering the Cloud Build pipeline.

Step 11 – Execute Cloud Build Pipeline

Description

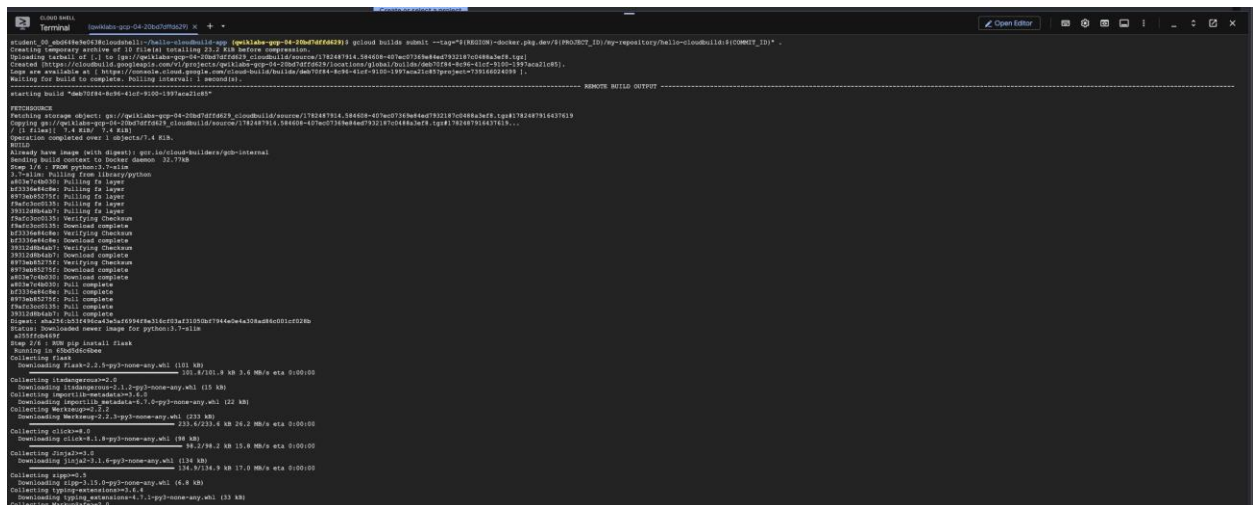
Cloud Build automatically cloned the repository, built the Docker image, and executed the CI/CD pipeline.



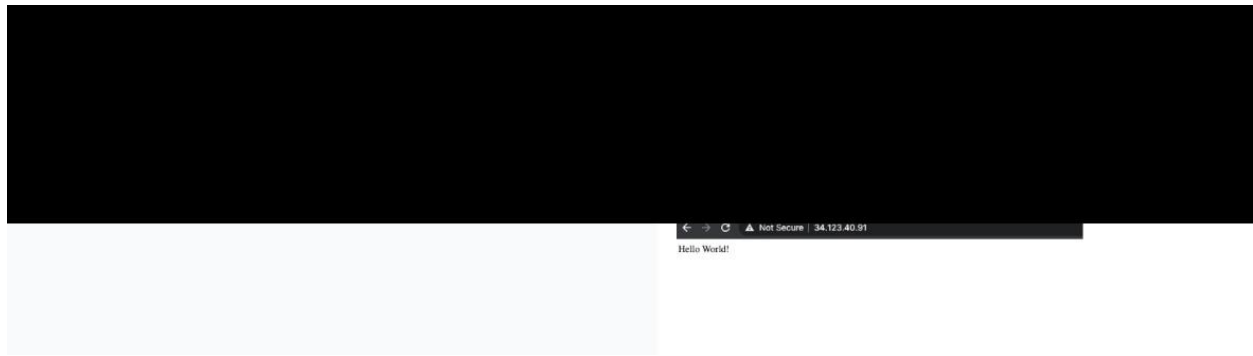
Step 12 – Push Docker Image

Description

The Docker image was successfully pushed into Google Artifact Registry.



Validated that the application was successfully deployed and accessible through the Kubernetes LoadBalancer.



8. Results

The project successfully demonstrated:

- GitHub Integration
- Cloud Build Automation
- Continuous Integration
- Continuous Delivery
- Docker Image Creation
- Artifact Registry Integration
- Kubernetes Deployment
- Automatic Build Trigger
- Automated Application Deployment
- GitOps Workflow

9. Challenges Faced

- Configuring GitHub authentication.
- Connecting Cloud Build GitHub App.
- Configuring Cloud Build triggers.
- Waiting for Docker image builds.
- Managing Kubernetes deployment manifests.
- Understanding GitOps workflow.
- Verifying deployment synchronization.

10. Learning Outcomes

This project provided practical experience in:

- GitOps
- Continuous Integration
- Continuous Delivery
- Cloud Build
- GitHub Integration
- Artifact Registry
- Google Kubernetes Engine
- Docker
- Kubernetes YAML
- Deployment Automation
- Cloud Native DevOps

11. Conclusion

This project successfully implemented a complete GitOps-based Continuous Delivery pipeline using Google Cloud Platform. Source code stored in GitHub automatically triggered Cloud Build, which built Docker images, pushed them into Artifact Registry, and deployed the latest application to Google Kubernetes Engine. The implementation demonstrated modern cloud-native DevOps practices including automated CI/CD, Infrastructure as Code, and GitOps deployment workflows commonly adopted in enterprise environments.

12. Source Code

Application Repository

Repository Name

hello-cloudbuild-app

GitHub Repository

<https://github.com/senthamizhan-matthew/hello-cloudbuild-app.git>

Description

This repository contains the Flask application source code, Dockerfile, Cloud Build configuration, Kubernetes deployment templates, and supporting files used to implement the GitOps Continuous Integration pipeline.

Environment Repository

Repository Name

hello-cloudbuild-env

GitHub Repository

<https://github.com/senthamizhan-matthew/hello-cloudbuild-env.git>

Description

This repository stores Kubernetes deployment manifests used by Cloud Build for Continuous Delivery to Google Kubernetes Engine. It maintains deployment configuration separately from application source code following GitOps best practices.